

# Chapter 12

## Inheritance and Polymorphism

# Topics

- Introduction to Inheritance
  - Why inheritance?
- Overloading and overriding/redefining a method
- Abstract classes
- Dynamic binding
  - Why languages support dynamic binding?
- Language examples
- Implementation of OO constructs
- Reflection

# Introduction

# Object-Oriented Programming

- Abstract data types
- Inheritance
  - Creating or deriving a new class/type using another class/type as a base
- Polymorphism
  - A method name means different things in different context

# Inheritance

- Inheritance allows new classes defined in terms of existing ones, i.e., by allowing them to inherit common parts
- Inheritance addresses two issues
  - Code reuse
    - All ADTs are independent, requires modification for any new use.
  - Program organization
    - All ADTs are at the same level, thus, it is impossible to organize a program to match the problem space.
- One disadvantage of inheritance for reuse:
  - Creates interdependencies among classes that complicate maintenance

# Inheritance in Java

```
public class Person
{
    private String name;

    public Person() {
        name = "No name yet.";
    }

    public Person(String initialName) {
        name = initialName;
    }

    public void setName(String newName) {
        name = newName;
    }

    public String getName() {
        return name;
    }

    @Override
    public String toString() {
        return "Name:\t" + this.name;
    }

    @Override
    public boolean equals(Object o) {
        if (o == this) {
            return true;
        }
        if (!(o instanceof Person)) {
            return false;
        }
        Person otherPerson = (Person) o;
        return (super.equals(otherPerson)
            && (this.name == otherPerson.name));
    }
}
```

@Override  
public int hashCode() {  
 return (name == null) ? 0 : name.hashCode();  
}

```
public class TestPersonStudent
{
    public static void main(String [] args)
    {
        Person p1 = new Person("John");
        Person p2 = new Student("Mary", 678);
        System.out.println(p1);
        System.out.println(p2);
    }
}
```

```
public class Student extends Person
{
    private int studentNumber;

    public Student() {
        super();
        studentNumber = 0;
    }

    public Student(String initialName, int initialStudentNumber) {
        super(initialName);
        studentNumber = initialStudentNumber;
    }

    public void reset(String newName, int newStudentNumber) {
        setName(newName);
        studentNumber = newStudentNumber;
    }

    public int getStudentNumber() {
        return studentNumber;
    }

    public void setStudentNumber(int newStudentNumber) {
        studentNumber = newStudentNumber;
    }

    @Override
    public String toString() {
        return super.toString() + "\nStudent Number:\t" + this.studentNumber;
    }

    @Override
    public boolean equals(Object o) {
        if (o == this) {
            return true;
        }
        if (!(o instanceof Student)) {
            return false;
        }
        Student otherStudent = (Student) o;
        return (super.equals(otherStudent)
            && (this.studentNumber == otherStudent.studentNumber));
    }
}
```

# Inheritance in c++

```
//Sample base class for c++ inheritance tutorial
```

```
class vehicle
{
protected:
    char colorname[20];
    int number_of_wheels;
public:
    vehicle();
    ~vehicle();
    void start();
    void stop();
    void run();
};
```

```
//Sample derived class for C++ inheritance tutorial
```

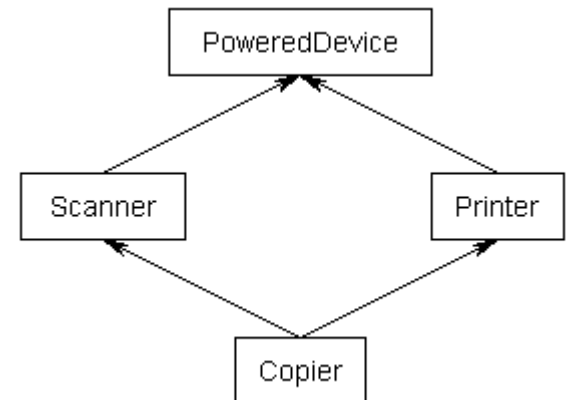
```
class Car: public vehicle
{
protected:
    char type_of_fuel;
public:
    Car();
};
```

# Some Related Concepts

- A class that inherits is a *derived class* or a *subclass*
- The class from which another class inherits is a parent class or *superclass* or *base class*
- Calls to methods are called *messages*
- The entire collection of methods of an object is called its *message protocol* or *message interface*

# Single and Multiple Inheritance

- **Single inheritance** → a new class is a subclass of a single parent class
- **Multiple inheritance** → a new class inherits from two or more classes
- Disadvantages of multiple inheritance:
  - Language and implementation complexity (in part due to name collisions and diamond problem)
  - Potential inefficiency - dynamic binding costs more with multiple inheritance (but not much)
- Advantage:
  - Sometimes it is extremely convenient and valuable



# **Method overloading and method overridden**

## **Abstract class**

# Method overriding

- The process of defining methods with the same name as defined in the **parent** class
  - Changing the **method implementation**, but not the method signatures
  - Provide an operation that is specific for objects of the derived class
- An overridden method may have a more specific **return type** as long as the new return type is derived from the overriding(parent) type.
- The **accessibility** must not be more restrictive than original method.
- The method must not throw **checked exceptions** of classes that are not possible for the original method.

# Method overloading

- The process of defining one or more methods with the same name, not necessarily of parent class.
- Change method implementation and parameter lists, but not the method name.
- A valid overload differs in the number or type of its parameters. Differences in parameter names are not significant. A different return type is permitted, but it is not sufficient by itself to distinguish an overloading method.

# Inheritance in Java

```
public class Person
{
    private String name;

    public Person() {
        name = "No name yet."; }

    public Person(String initialName) {
        name = initialName; }

    public void setName(String newName) {
        name = newName;}

    public String getName() {
        return name; }

    @Override
    public String toString() {
        return "Name:\t" + this.name; }

    @Override
    public boolean equals(Object o) {
        if (o == this) {
            return true; }
        if (!(o instanceof Person)) {
            return false; }
        Person otherPerson = (Person) o;
        return (super.equals(otherPerson)
            && (this.name == otherPerson.name))
    }
}
```

```
public class TestPersonStudent
{
    public static void main(String [] args)
    {
        Person p1 = new Person("John");
        Person p2 = new Student("Mary", 678);
        System.out.println(p1);
        System.out.println(p2);
    }
}
```

```
public class Student extends Person
{
    private int studentNumber;

    public Student(){
        super();
        studentNumber = 0;}

    public Student(String initialName, int initialStudentNumber){
        super(initialName);
        studentNumber = initialStudentNumber;}

    public void reset(String newName, int newStudentNumber) {
        setName(newName);
        studentNumber = newStudentNumber; }

    public int getStudentNumber() {
        return studentNumber; }

    public void setStudentNumber(int newStudentNumber){
        studentNumber = newStudentNumber; }

    @Override
    public String toString() {
        return super.toString() + "\nStudent Number:\t" + this.studentNumber; }

    @Override
    public boolean equals(Object o) {
        if (o == this) {
            return true; }
        if (!(o instanceof Student)) {
            return false;
        }
        Student otherStudent = (Student) o;
        return (super.equals(otherStudent)
            && (this.studentNumber == otherStudent.studentNumber)); }
}
```

# Method Overloading (cont.)

- You should restrict your use of overloaded method names on situations where the methods really are performing the same basic function with different data.
- The language treats methods with overloaded names as totally different methods and can coexist in a single class.
- When C++/java encounters a call to an overloaded method, it simply executes the version of the method whose parameters match the arguments used in the call.
- Automatic type conversion only if no exact match is found.

```

#include <iostream>

using namespace std;

class Homer{
public:
    char doh(char c){
        cout << "doh(char)" << endl;
        return 'd'; }
    float doh(float f){
        cout << "doh(float)" << endl;
        return 1.0f; }
};

class Milhouse{};

class Bart: public Homer{
public:
    void doh(Milhouse & m){
        cout << "doh(Milhouse)" << endl; }
};

int main(){
    Bart b;
    b.doh(1);
    b.doh('x');
    Milhouse m1;
    b.doh(m1);
    return 0;
}

```

# Abstract class

- An *abstract method* is one that does not include a definition (it only defines a protocol)
- Generally, an *abstract class* includes at least one abstract/pure virtual method
- An abstract class cannot be instantiated

```

#include <iostream>

using namespace std;

// Base class
class Shape
{
public:
    // pure virtual function providing interface framework.
    virtual int getArea() = 0;
    void setWidth(int w)
    {
        width = w;
    }
    void setHeight(int h)
    {
        height = h;
    }
protected:
    int width;
    int height;
};

// Derived classes
class Rectangle: public Shape
{
public:
    int getArea()
    {
        return (width * height);
    }
};

class Triangle: public Shape
{
public:
    int getArea()
    {
        return (width * height)/2;
    }
};

```

```

int main(void)
{
    Rectangle Rect;
    Triangle Tri;

    Rect.setWidth(5);
    Rect.setHeight(7);
    // Print the area of the object.
    cout << "Total Rectangle area: " << Rect.getArea() << endl;

    Tri.setWidth(5);
    Tri.setHeight(7);
    // Print the area of the object.
    cout << "Total Triangle area: " << Tri.getArea() << endl;

    return 0;
}

```

Abstract classes can carry a **functional "payload"** that numerous subclasses inherit and can use directly

# **Dynamic Binding Or Dynamic Dispatch Or Polymorphism**

# Dynamic Binding

- A binding is an association, for example, a variable is bound to a data type
- Static binding: the binding occurs before run time and remains unchanged throughout program execution.
- Dynamic binding: the binding occurs during run time or can change in the course of program execution.

# Polymorphism

- From the Greek meaning “many forms”
- The same program instruction adapts to mean different things in different contexts.
  - A method/function name, used as an instruction, produces results that depend on the class of the object that used the method.

# Dynamic Binding and polymorphism

- A *polymorphic variable* can be defined in a class that is able to reference (or point to) objects of the class and objects of any of its descendants.
- **When a class hierarchy includes classes that override methods and such methods are called through a polymorphic variable, the binding to the correct method will be dynamic.**
- Allows software systems to be more easily **extended** during both development and maintenance

# An Example of Polymorphism

```
public class TestPersonStudent
{
    public static void main(String [] args)
    {
        Person p1 = new Person("John");
        Person p2 = new Student("Mary", 678);
        System.out.println(p1);
        System.out.println(p2);
    }
}
```

```

//: C15:Instrument2.cpp
// Inheritance & upcasting
#include <iostream>
using namespace std;
enum note { middleC, Csharp, Eflat }; // Etc.

```

```

class Instrument {
public:
virtual → void play(note) const {
    cout << "Instrument::play" << endl;
}
};

```

```

// Wind objects are Instruments
// because they have the same interface:
class Wind : public Instrument {
public:
    // Redefine interface function:
    void play(note) const {
        cout << "Wind::play" << endl;
    }
};

```

```

void tune(Instrument& i) {
    // ...
    i.play(middleC);
}

```

```

int main() {
    Wind flute;
    tune(flute); // Upcasting →
} ///:~

```

Taking the **address of an object** (either a pointer or a reference) and treating it as the **address of the base type**

## Why dynamic binding? Extensibility

# Extensible

- A program is *extensible* if you can add new functionality by inheriting new data types from the common base class. The functions that manipulate the **base-class interface** will not need to be changed at all to accommodate the new classes.

# An extended program

```
#include <iostream>
using namespace std;
enum note { middleC, Csharp, Cflat }; // Etc.
```

```
class Instrument {
public:
    virtual void play(note) const {
        cout << "Instrument::play" << endl;
    }
    virtual char* what() const {
        return "Instrument";
    }
    // Assume this will modify the object:
    virtual void adjust(int) {}
};
```

```
class Wind : public Instrument {
public:
    void play(note) const {
        cout << "Wind::play" << endl;
    }
    char* what() const { return "Wind"; }
    void adjust(int) {}
};
```

```
class Percussion : public Instrument {
public:
    void play(note) const {
        cout << "Percussion::play" << endl;
    }
    char* what() const { return "Percussion"; }
    void adjust(int) {}
};
```

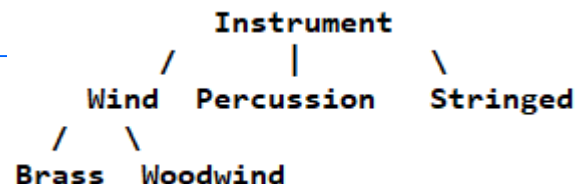
```
class Stringed : public Instrument {
public:
    void play(note) const {
        cout << "Stringed::play" << endl;
    }
    char* what() const { return "Stringed"; }
    void adjust(int) {}
};
```

```
class Brass : public Wind {
public:
    void play(note) const {
        cout << "Brass::play" << endl;
    }
    char* what() const { return "Brass"; }
};
```

```
class Woodwind : public Wind {
public:
    void play(note) const {
        cout << "Woodwind::play" << endl;
    }
    char* what() const { return "Woodwind"; }
};
```

```
// Identical function from before:
void tune(Instrument& i) {
    // ...
    i.play(middleC);
}

// New function:
void f(Instrument& i) { i.adjust(1); }
```



# Inheritance in Java

# Inheritance in Java

- **Overview**

- All Java classes must be subclasses of the *Object* class, or some class that is descendent of *Object*
- All Java objects are explicit heap-dynamic
- Java supports only **single inheritance**

- A derived class inherits

- all of the *public* and *protected* members of its parent, no matter what package the subclass is in.
- A derived class does not inherit the *private* members of its parent class.
- Constructors are not members, so they are not inherited by subclasses

# Initialization: Constructors

- Java (and C++) automatically inserts calls to the base-class constructor in the derived-class constructor.

```
//: Cartoon.java
// Constructor calls during inheritance

class Art {
    Art() {
        System.out.println("Art constructor");
    }
}

class Drawing extends Art {
    Drawing() {
        System.out.println("Drawing constructor");
    }
}

public class Cartoon extends Drawing {
    Cartoon() {
        System.out.println("Cartoon constructor");
    }
    public static void main(String[] args) {
        Cartoon x = new Cartoon();
    }
} ///:~
```

# Initialization: Constructors

- Java (and C++) automatically inserts calls to the base-class constructor in the derived-class constructor.

```
//: Cartoon.java  
// Constructor calls during inheritance
```

```
class Art {  
    Art() {  
        System.out.println("Art constructor");  
    }  
}
```

```
class Drawing extends Art {  
    Drawing() {  
        System.out.println("Drawing  
constructor");  
    }  
}
```

```
public class Cartoon extends Drawing {  
    Cartoon() {  
        System.out.println("Cartoon  
constructor");  
    }  
    public static void main(String[] args) {  
        Cartoon x = new Cartoon();  
    }  
} ///:~
```

The output for this program shows the automatic calls:  
Art constructor  
Drawing constructor  
Cartoon constructor

# Initialization: Constructors (cont.)

- To call a base-class constructor that has an argument → explicitly write the calls to the base-class constructor using the **super** keyword and the appropriate argument list.
- **super** *must* be the first thing you do in the derived-class constructor

```
//: Chess.java  
// Inheritance, constructors and arguments
```

```
class Game {  
    Game(int i) {  
        System.out.println("Game constructor");  
    }  
}  
  
class BoardGame extends Game {  
    BoardGame(int i) {  
        super(i);  
        System.out.println("BoardGame constructor");  
    }  
}  
  
public class Chess extends BoardGame {  
    Chess() {  
        super(11);  
        System.out.println("Chess constructor");  
    }  
    public static void main(String[] args) {  
        Chess x = new Chess();  
    }  
} ///:~
```

The output for this program

??

# Protected

- The **protected** keyword says “This is **private** as far as the class user is concerned, but available to anyone who inherits from this class or anyone else in the same **package**.” That is, **protected** in Java is automatically “friendly.”

# The final keyword

- Final data
  - With a **primitive**, **final** makes the *value* a **constant**, but with an **object handle**, **final** makes the **handle** a constant.
  - A field that is both **static** and **final** has only one piece of storage that cannot be changed.

```
//: FinalData.java
// The effect of final on fields
```

```
class Value {
    int i = 1;
}
```

**I3: public** so they're usable outside the package,  
**static** to emphasize that there's only one,  
and **final** to say that it's a constant.

```
public class FinalData {
    // Typical public constant:
    public static final int I3 = 39;
```

```
    // final objects
    final Value v2 = new Value();
```

```
    public static void main(String[] args) {
        FinalData fd1 = new FinalData();
```

```
        fd1.v2.i++; // Object isn't constant!
```

```
        //! fd1.v2 = new Value(); // Error: Can't change handle
```

```
    }
} ///:~
```

**v2:v2** is **final**-- doesn't mean that you can't change its value.  
However, you cannot re-bind **v2** to a new object, precisely because  
it's **final**.

# The final keyword

- Final methods
  - Prevent methods to be overridden
  - Allow the compiler to turn any calls to that method into *inline* calls
- Final class
  - The class cannot be inherited
  - All methods in a **final** class are implicitly **final**, since there's no way to override them.

# Multiple inheritance

- Realize through interface

```
public abstract class Keyed
{
    ...
}

public interface Sorted
{
    ...
}

public class KeyedIndex extends Keyed
    implements Sorted
{
    ...
}
```

# Inheritance in C++

# Inheritance in C++

- General Characteristics:
  - Mixed typing system → Add objects to a complete typing system
  - A class need not be the subclass of some class
- What a derived class doesn't inherit in C++
  - The base class's constructors and destructor
  - The base class's assignment operator
  - The base class's friends

# Public vs private inheritance

- public keyword → the public and protected members inherited from the base class are also public and protected in the derived class
- private keyword → **public and protected** members inherited from the base class become *private* in the derived class

```
class base_class {  
    private:  
        int a;  
        float x;  
    protected:  
        int b;  
        float y;  
    public:  
        int c;  
        float z;  
};
```

```
class subclass_1 : public base_class { ... };  
//      In this one, b and y are protected and  
//      c and z are public
```

```
class subclass_2 : private base_class { ... };  
//      In this one, b, y, c, and z are private,  
//      and no derived class of subclass_2 has access to any  
//      member of base_class
```

The data members a and x in base\_class are not accessible in either subclass\_1 or subclass\_2.

```

class Stack1: public Single_linked_list{
public:
    Stack1(){}
    void push(int value){
        Single_linked_list::insert_at_head(value);
    }
    int pop(){
        return Single_linked_list::remove_at_head();
    }
};

```

```

class Single_linked_list{
private:
    class Node{
    public:
        node *link;
        int contents;
    };
    node *head;
public:
    Single_linked_list(){head = 0};
    void insert_at_head(int);
    void insert_at_end(int);
    int remove_at_head();
    int empty();
};

```

## Public vs private inheritance example

```

class Stack2: private Single_linked_list
{
public:
    Stack2(){}
    void push(int value){
        Single_linked_list::insert_at_head(value);
    }

    int pop(){
        return Single_linked_list::remove_at_head();
    }
    using Single_linked_list:: empty;
};

```

# Reexportation in C++

- Under private class derivation, no member of the parent class is implicitly visible to the instances of the derived class.
- A member that is not accessible in a subclass (because of private derivation) can be declared to be visible there using the scope resolution operator (`::`), e.g.,

```
class subclass_3 : private base_class {  
    base_class :: c;  
    ...  
}
```

# Initialization

- The derived class's constructor automatically calls the "no argument" base class constructor(s)
- For a non-void constructor, specify which base-class constructor gets called, as part of the definition of the derived-class's constructor

```
///  
// Constructor calls during inheritance
```

```
#include <iostream>  
using namespace std;
```

```
class Art {  
public:  
    Art() {  
        cout << "Art constructor\n";  
    }  
};
```

```
class Drawing: public Art {  
public:  
    Drawing() {  
        cout << "Drawing constructor\n";  
    }  
};
```

```
class Cartoon:public Drawing {  
public:  
    Cartoon() {  
        cout << "Cartoon constructor\n";  
    }  
};
```

```
int main()  
{  
    Cartoon x;  
}
```

Constructor  
calls with  
inheritance

# Polymorphism

- Member functions can be marked as polymorphic using the **virtual** keyword (**virtual functions**)
- Calls to such functions get dispatched according to the **actual type of the object** which owns them, rather than the apparent type of the pointer (or reference) through which they are invoked

```

//: C15:Instrument2.cpp
// Inheritance & upcasting
#include <iostream>
using namespace std;
enum note { middleC, Csharp, Eflat }; // Etc.

```

```

class Instrument {
public:
virtual → void play(note) const {
    cout << "Instrument::play" << endl;
}
};

```

```

// Wind objects are Instruments
// because they have the same interface:
class Wind : public Instrument {
public:
    // Redefine interface function:
    void play(note) const {
        cout << "Wind::play" << endl;
    }
};

```

```

void tune(Instrument& i) {
    // ...
    i.play(middleC);
}

```

```

int main() {
    Wind flute;
    tune(flute); // Upcasting →
} //:~

```

Taking the **address of an object** (either a pointer or a reference) and treating it as **the address of the base type**

## Why dynamic binding? Extensibility

# Multiple inheritance

- Multiple inheritance is supported
  - If there are two inherited members with the same name, they can both be referenced using the scope resolution operator

# Objects are different

- Upcasting deals only with addresses.
- If the compiler has an object, it knows the exact type and therefore (in C++) will not use late binding for any function calls – or at least, the compiler doesn't *need* to use late binding.
- For efficiency's sake, most compilers will perform early binding when they are making a call to a virtual function for an object because they know the exact type.

```

// Early binding & virtual functions
#include <iostream>
#include <string>
using namespace std;

class Pet {
public:
    virtual string speak() const { return ""; }
};

class Dog : public Pet {
public:
    string speak() const { return "Bark!"; }
};

int main() {
    Dog ralph;
    Pet* p1 = &ralph;
    Pet& p2 = ralph;
    Pet p3;
    // Late binding for both:
    cout << "p1->speak() = " << p1->speak() << endl;
    cout << "p2.speak() = " << p2.speak() << endl;
    // Early binding (probably):
    cout << "p3.speak() = " << p3.speak() << endl;
} ///:~

```

**p1** and **p2** can represent the address of a **Pet** or something derived from **Pet**, so the virtual mechanism must be used.

When calling **p3.speak()** there's no ambiguity. Thus, early binding is probably used.

# Implementing OO Constructs

# Implementing OO Constructs

- Two interesting and challenging parts
  - Storage structures for instance variables
  - Dynamic binding of messages to methods

# Instance Data Storage

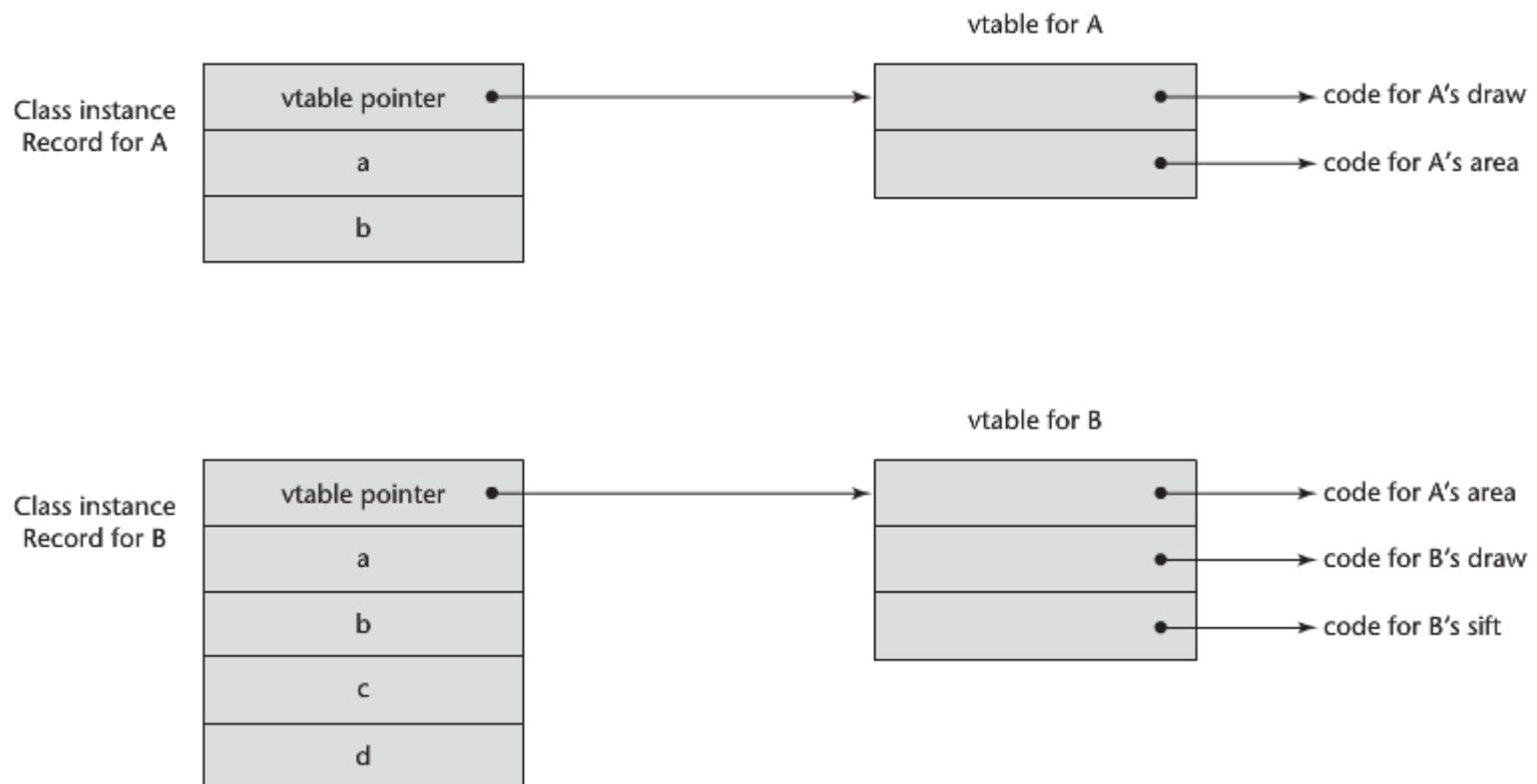
- **Class instance records** (CIRs) store the state of an object
  - Static (built at compile time)
- If a class has a parent, the subclass instance variables are added to the parent CIR (The parent's CIR itself is not changed.)

# Dynamic Binding of Methods Calls

- Calls to dynamically bound methods can be connected to the corresponding code thru a pointer, i.e., vtable pointer, in the CIR
- The storage structure is sometimes called *virtual method tables* (vtable)
- Method calls can be represented as offsets from the beginning of the vtable

```
public class A {
    public int a, b;
    public void draw() { ... }
    public int area() { ... }
}
```

```
public class B extends A {
    public int c, d;
    public void draw() { ... }
    public void sift() { ... }
}
```



How dynamic binding works?

```

class A {
public:
    int a;
    virtual void fun() { ... }
    virtual void init() { ... }
};

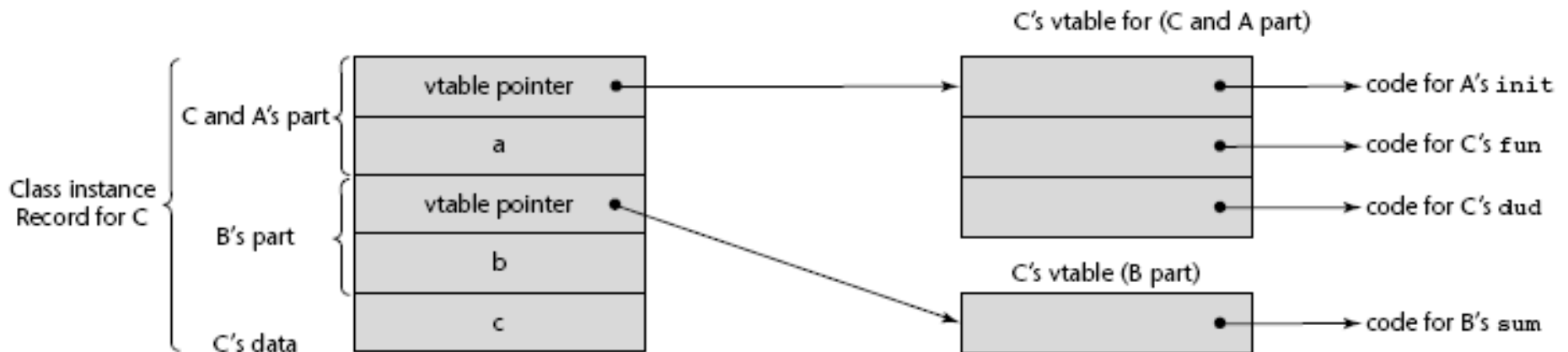
class B {
public:
    int b;
    virtual void sum() { ... }
};

```

```

class C : public A, public B {
public:
    int c;
    virtual void fun() { ... }
    virtual void dud() { ... }
};

```



**Figure 12.3**

An example of a subclass CIR with multiple parents

# Language design issue

- In some object-oriented languages, dynamic/late binding always take place and it is not be an option, and the user shouldn't have to know about it.
- Dynamic/Late binding does require both code space and execution time. In other languages, late binding is an option.

# Virtual or not?

- When designing your classes, however, you shouldn't be worrying about efficiency tuning. If you're going to use polymorphism, use virtual functions everywhere.
- You might get better size and speed efficiency is because you may design a C++ program in a smaller, faster way than you would using C.

# Reflection

--reflection provides the possibility of late binding of calls to methods that are outside the inheritance hierarchy of the calling code.

# Reflection

- A programming language that supports reflection allows its programs to have **runtime access to their types and structure (metadata)**, and to be able to dynamically **modify their behavior**
- The process of a program examining its metadata is called *introspection*
- Interceding in the execution of a program is called *intercession*

# Reflection in Java

```
import java.lang.reflect.*;

class Reflect {
    public static void callDraw(Object birdObj) {
        Class cls = birdObj.getClass();
        try {
            // Find the draw method of the given class
            Method method = cls.getMethod("draw");
            // Dynamically call the method
            method.invoke(birdObj);
        }
        // In case the given class does not support draw
        catch (NoSuchMethodException e) {
            throw new IllegalArgumentException (
                cls.getName() + "does not support draw");
        }
        // In case the callDraw cannot call draw
        catch (IllegalAccessException e) {
            throw new IllegalArgumentException (
                "Insufficient access permissions to call" +
                "draw in class " + cls.getName());
        }
        // In case draw throws an exception
        catch (InvocationTargetException e) {
            throw new RuntimeException(e);
        }
    }
}
```

# Reflection in Java/2

```
]class Bird1 {  
-   public void draw() {  
-       System.out.println("This is draw from Bird1");  
-   }  
-}  
[class Bird2 {  
-   public void draw() {  
-       System.out.println("This is draw from Bird2");  
-   }  
-}  
[class Bird3 {  
-   public void draw() {  
-       System.out.println("This is draw from Bird3");  
-   }  
-}  
[public class ReflectTest {  
-   public static void main(String[] args) {  
-       Object[] birdList = new Object[3];  
-       birdList[0] = new Bird1();  
-       birdList[1] = new Bird2();  
-       birdList[2] = new Bird3();  
-       Reflect.callDraw(birdList[2]);  
-       Reflect.callDraw(birdList[0]);  
-       Reflect.callDraw(birdList[1]);  
-   }  
-}
```

# Summary

- OO programming involves three fundamental concepts: ADTs, inheritance, dynamic binding
- Major design issues: exclusivity of objects, subclasses and subtypes, type checking and polymorphism, single and multiple inheritance, dynamic binding, explicit and implicit de-allocation of objects, and nested classes
- C++ has two distinct type system (hybrid)
- Java is not a hybrid language like C++; it supports only OO programming

```

#include <iostream>
#include <cstring>
using namespace std;

class Setter
{
public:
    void Set(char*& variable, char* value)
    {
        variable = new char[strlen(value)+1];
        strcpy(variable,value);
    }
};

class Name: private Setter
{
public:
    Name(void) { myName = 0; }
    ~Name(void) { delete[] myName; }

    void SetName(char* n)
    { Set(myName,n); }

    void Print(void)
    { cout << myName << endl; }

private:
    char* myName;
};

```

```

class Contact: public Name
{
public:
    Contact(void) { myAddress = 0; }
    ~Contact(void) { delete[] myAddress; }

    void SetAddress(char* c){}
    { Set(myAddress,c); }

    void Print(void)
    {
        Name::Print();
        cout << myAddress << endl;
    }

private:
    char* myAddress;
};

int main()
{
    Contact c;

    c.SetName("John McClane");

    c.SetAddress("137th floor, Nakatome Towers");

    c.Print();
    return 0;
}

```

Exercise 3: Explain the output/behavior during program execution.

```

1  //: C14:NameHiding.cpp
2
3  #include <iostream>
4  #include <string>
5  using namespace std;
6
7  class Base {
8  public:
9      int f() const {
10         cout << "Base::f()\n";
11         return 1;
12     }
13     int f(string) const { return 1; }
14     void g() {}
15 };
16
17 class Derived1 : public Base {
18 public:
19     void g() const {}
20 };
21
22 class Derived2 : public Base {
23 public:
24     // Redefinition:
25     int f() const {
26         cout << "Derived2::f()\n";
27         return 2;
28     }
29 };
30
31 class Derived3 : public Base {
32 public:
33     // Change return type:
34     void f() const { cout << "Derived3::f()\n"; }
35 };
36
37 class Derived4 : public Base {
38 public:
39     // Change argument list:
40     int f(int) const {
41         cout << "Derived4::f()\n";
42         return 4;
43     }
44 };
45
46 int main() {
47     string s("hello");
48     Derived1 d1;
49     int x = d1.f();
50     d1.f(s);
51     Derived2 d2;
52     x = d2.f();
53     d2.f(s);
54     Derived3 d3;
55     x = d3.f();
56     Derived4 d4;
57     x = d4.f();
58     x = d4.f(1);
59 } ///:~
60

```

**Exercise 4. Explain the output/behavior during program execution.**